SPÉCIFICATION & IMPLÉMENTATION COMPLÈTE • PIPELINE AUDIO TEMPS RÉEL

Analyseur Vocal de Surveillance (Voice Analyzer)

Architecture de bout en bout : Capture VAD, Transcription Whisper, Extraction d'Entités (NER), Corrélation Tactique et Interface FUI Temps Réel

1. Architecture Globale et Structure du Projet

Ce document rassemble l'implémentation complète et fonctionnelle de l'analyseur vocal de surveillance. Le système intercepte un flux audio en continu (microphone ou piste d'écoute préenregistrée), segmente la parole active, transcrit les énoncés à la volée, extrait les entités critiques et met à jour une interface tactique (FUI) reproduisant le comportement visuel de l'écran d'origine : surlignage vert des mots-clés dans le flux, liaison vectorielle pointillée vers la matrice de corrélation et verrouillage automatique (`Locked`) des cibles et zones géographiques.

Module / Fichier	Technologies Clés	Rôle dans le Pipeline
`requirements.txt`	PyPI / Environnement Python 3.10+	Déclaration des dépendances audio, apprentissage profond, NLP et serveur web asynchrone.
`intel_engine.py`	`spaCy` / Règles NER & Graphe de corrélation	Détection des entités (Cibles, Lieux, Menaces, Actifs, Délais), alimentation de la grille 6×2 et calcul du verrouillage `Summary` .
`audio_transcriber.py`	`sounddevice` , `Silero VAD` , `faster-whisper`	Capture micro en streaming (16 kHz), découpage des phrases actives et transcription faible latence (inclut un simulateur de test).
`server.py`	`FastAPI` , `Uvicorn` , `WebSockets`	Orchestrateur temps réel : pousse les segments annotés et l'état du tableau de bord vers le navigateur en JSON.
`static/index.html`	HTML5, CSS3 Tactique, SVG Dynamique, JS	Interface FUI sombre avec transcription animée, badges verts, flèches pointillées dynamiques et panneaux `Summary` / `Voice Analyzer` .

1.1 Arborescence et fichier requirements.txt

Créez un dossier de travail contenant l'arborescence suivante, puis installez les dépendances listées dans `requirements.txt` :

```
voice-analyzer/
├── requirements.txt
├── intel_engine.py
├── audio_transcriber.py
├── server.py
└── static/
    └── index.html
```

Fichier : requirements.txt

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
websockets==13.0.1
faster-whisper==1.0.3
torch==2.4.0
torchaudio==2.4.0
sounddevice==0.5.0
numpy==1.26.4
spacy==3.7.6
```

2. Moteur d'Extraction NER et de Corrélation (Étapes 3 & 4)

Le module `intel_engine.py` combine la reconnaissance d'entités nommées (via `spaCy` et un dictionnaire de règles tactiques personnalisables) avec une mémoire d'investigation. Chaque fois qu'un segment de texte arrive du transcripteur, le moteur repère la position exacte des mots-clés pour leur attribuer un identifiant unique, remplit les cellules de la grille supérieure et verrouille la cible principale (`Subject: AMIRI`) ainsi que les coordonnées géographiques (`UKRAINE` , `BLACK SEA`).

Fichier : intel_engine.py (Partie 1/2 — Détection d'entités et base de connaissances)

```
import re
from typing import Dict, List, Any

class TacticalIntelEngine:
    """
    Moteur d'extraction d'entités (NER) et de corrélation SIGINT.
    Associe les mots-clés détectés aux profils de cibles et met à jour la matrice FUI.
    """
    def __init__(self):
        # Catalogue de cibles connues et règles de corrélation
        self.knowledge_base = {
            "alexander": {
                "canonical": "ALEXANDER",
                "category": "SUBJECT_ALIAS",
                "linked_subject": "AMIRI",
                "grid_slot": (0, 0),
                "paired_slot": (0, 1, "AMIRI")
            },
            "security": {
                "canonical": "SECURITY",
                "category": "SECTOR",
                "grid_slot": (1, 0),
                "paired_slot": (1, 1, "UKRAINE")
            },
            "cat burglar": {
                "canonical": "CAT BURGLAR",
                "category": "OPERATIVE",
                "grid_slot": (2, 0),
                "paired_slot": (2, 1, "BLACK SEA")
            },
            "flying fowl": {
                "canonical": "FLYING FOWL",
                "category": "CODENAME",
                "grid_slot": (3, 0),
                "paired_slot": None
            },
            "72 hours": {
                "canonical": "72 HOURS",
                "category": "TIMELINE",
                "grid_slot": (4, 0),
                "paired_slot": None
            }
        }

        # État de la matrice 6 lignes x 2 colonnes (affichée en haut à droite)
        self.grid_rows: List[List[str]] = [["", ""] for _ in line_range(6)]
        self.active_highlight_cell = (3, 0)  # Cellule hachurée active (ex: FLYING
FOWL)
        self.subject = "SEARCHING..."
        self.location_locked = False
        self.id_locked = False
        self.primary_locations: List[str] = []
        self.line_counter = 0

    def reset(self):
        self.grid_rows = [["", ""] for _ in range(6)]
        self.subject = "SEARCHING..."
        self.location_locked = False
```

Fichier : intel_engine.py (Partie 2/2 — Analyse de segment et verrouillage Summary)

```
    def analyze_utterance(self, text: str) -> Dict[str, Any]:
        """
        Analyse une phrase transcrite, extrait les entités à surligner en vert
        et met à jour l'état du panneau Summary et de la grille de corrélation.
        """
        self.line_counter += 1
        utterance_id = f"utt-{self.line_counter}"
        detected_spans = []
        lower_text = text.lower()

        for pattern, meta in self.knowledge_base.items():
            for match in re.finditer(r'\b' + re.escape(pattern) + r'\b', lower_text):
                start, end = match.span()
                raw_word = text[start:end]
                row_idx, col_idx = meta["grid_slot"]

                # Mise à jour de la cellule gauche de la grille
                self.grid_rows[row_idx][col_idx] = meta["canonical"]

                # Mise à jour de la cellule droite corrélée (si applicable)
                if meta["paired_slot"]:
                    p_row, p_col, p_val = meta["paired_slot"]
                    self.grid_rows[p_row][p_col] = p_val
                    if p_val in ("UKRAINE", "BLACK SEA") and p_val not in
self.primary_locations:
                        self.primary_locations.append(p_val)

                # Verrouillage du sujet principal
                if meta.get("linked_subject"):
                    self.subject = meta["linked_subject"]
                    self.id_locked = True

                if len(self.primary_locations) >= 2:
                    self.location_locked = True

                detected_spans.append({
                    "span_id": f"kw-{self.line_counter}-{start}",
                    "text": raw_word,
                    "start": start,
                    "end": end,
                    "canonical": meta["canonical"],
                    "category": meta["category"],
                    "target_cell": f"cell-{row_idx}-{col_idx}",
                    "draw_arrow": (meta["canonical"] == "FLYING FOWL")
                })

        # Tri des entités par ordre d'apparition dans la phrase
        detected_spans.sort(key=lambda x: x["start"])

        return {
            "utterance_id": utterance_id,
            "raw_text": text,
            "entities": detected_spans,
            "state": {
                "grid": self.grid_rows,
                "active_cell": f"cell-{self.active_highlight_cell[0]}-
{self.active_highlight_cell[1]}",
                "summary": {
```

3. Capture Audio VAD et Transcription Temps Réel (Étapes 1 & 2)

Le module `audio_transcriber.py` gère l'acquisition sonore en temps réel. Il inclut deux modes de fonctionnement sélectionnables par variable d'environnement : un **mode Microphone Live** qui utilise `sounddevice`, `Silero VAD` et `faster-whisper`, ainsi qu'un **mode Simulation Tactique** qui rejoue la transcription exacte visible sur l'écran de surveillance pour tester immédiatement le rendu visuel et les flèches vectorielles sans matériel audio dédié.

> **Fichier : audio_transcriber.py (Partie 1/2 — Mode Simulation et Initialisation Whisper/VAD)**

```
import asyncio
import queue
import numpy as np
from typing import AsyncGenerator

# Script de test reproduisant fidèlement l'interception de l'écran Voice Analyzer
SIMULATED_INTERCEPT_LINES = [
    "We need to talk.",
    "I just spoke to Alexander.",
    "Wasn't easy getting an answer out of him.",
    "He's in a terrible mood.",
    "You know he prides himself on his security.",
    "Some cat burglar got in last night, gassed the house.",
    "Cleaned out all his cash and jewellery.",
    "On leaving, Alexander's personal guards stopped him.",
    "He hospitalized two of his guards.",
    "One with a broken nose, three cracked ribs.",
    "The other one, a bullet in the foot.",
    "They're going to carry on. I'm waiting on confirmation.",
    "They're moving Flying Fowl in 72 hours.",
    "They should be ready."
]

class AudioStreamTranscriber:
    """
    Pipeline Étapes 1 & 2 : Capture audio 16kHz + Silero VAD + Faster-Whisper.
    """
    def __init__(self, use_microphone: bool = False, model_size: str = "base.en"):
        self.use_microphone = use_microphone
        self.model_size = model_size
        self.sample_rate = 16000
        self.audio_queue: "queue.Queue[np.ndarray]" = queue.Queue()
        self.whisper_model = None
        self.vad_model = None

        if self.use_microphone:
            self._init_models()

    def _init_models(self):
        import torch
        from faster_whisper import WhisperModel
        # Chargement de Silero VAD pour isoler uniquement la voix active
        self.vad_model, _ = torch.hub.load(
            repo_or_dir="snakers4/silero-vad",
            model="silero_vad",
            force_reload=False
        )
        # Chargement de Faster-Whisper en quantification INT8 pour faible latence
CPU/GPU
        self.whisper_model = WhisperModel(
            self.model_size,
            device="cpu",
            compute_type="int8"
        )
```

Fichier : audio_transcriber.py (Partie 2/2 — Boucle d'écoute VAD et Générateur Async)

```
    def _audio_callback(self, indata, frames, time_info, status):
        """Callback appelé en continu par sounddevice à chaque bloc audio."""
        self.audio_queue.put(indata[:, 0].copy())

    async def stream_transcriptions(self) -> AsyncGenerator[str, None]:
        """
        Génère les phrases transcrites au fil de l'eau (soit via le simulateur,
        soit depuis le microphone filtré par Silero VAD).
        """
        if not self.use_microphone:
            for line in SIMULATED_INTERCEPT_LINES:
                await asyncio.sleep(1.35)
                yield line
            return

        import sounddevice as sd
        import torch

        speech_buffer = []
        silence_frames = 0
        chunk_samples = 512  # 32 ms à 16 kHz

        with sd.InputStream(
            samplerate=self.sample_rate,
            channels=1,
            dtype="float32",
            blocksize=chunk_samples,
            callback=self._audio_callback
        ):
            while True:
                await asyncio.sleep(0.01)
                while not self.audio_queue.empty():
                    chunk = self.audio_queue.get_nowait()
                    tensor_chunk = torch.from_numpy(chunk)
                    speech_prob = self.vad_model(tensor_chunk, self.sample_rate).item()

                    if speech_prob > 0.5:
                        speech_buffer.append(chunk)
                        silence_frames = 0
                    elif len(speech_buffer) > 0:
                        speech_buffer.append(chunk)
                        silence_frames += 1

                        # Si ~600 ms de silence après une phrase, on lance Whisper
                        if silence_frames > 18:
                            full_audio = np.concatenate(speech_buffer)
                            speech_buffer.clear()
                            silence_frames = 0

                            segments, _ = self.whisper_model.transcribe(
                                full_audio,
                                beam_size=1,
                                language="en",
                                vad_filter=False
                            )
                            text = " ".join(seg.text.strip() for seg in
segments).strip()
                            if text:
                                yield text
```

4. Backend de Streaming Temps Réel (Étape 5)

Le serveur `server.py` utilise `FastAPI` et un canal `WebSocket` (`/ws/intercept`) pour relier le pipeline Python à l'interface graphique. Dès que le client ouvre l'interface dans son navigateur, le serveur démarre le flux d'interception et transmet chaque phrase accompagnée de ses métadonnées d'entités et de l'état de verrouillage.

Fichier : server.py

```
import os
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse

from intel_engine import TacticalIntelEngine
from audio_transcriber import AudioStreamTranscriber

app = FastAPI(title="SIGINT Voice Analyzer Server")

# Montage du dossier contenant l'interface FUI
app.mount("/static", StaticFiles(directory="static"), name="static")

@app.get("/")
async def serve_ui():
    return FileResponse("static/index.html")

@app.websocket("/ws/intercept")
async def websocket_intercept(websocket: WebSocket):
    await websocket.accept()
    use_mic = os.getenv("USE_MIC", "0") == "1"

    engine = TacticalIntelEngine()
    transcriber = AudioStreamTranscriber(use_microphone=use_mic)

    try:
        async for utterance in transcriber.stream_transcriptions():
            payload = engine.analyze_utterance(utterance)
            await websocket.send_json(payload)
    except WebSocketDisconnect:
        print("Client déconnecté du flux d'interception.")
```

5. Interface Visuelle Tactique FUI (Étape 6)

Le fichier `static/index.html` implémente l'interface complète. Elle intègre la typographie monospace phosphorée, le surlignage vert vif (`#38ef7d`) sur les entités détectées dans la colonne de gauche, la grille de corrélation 6×2 en haut à droite avec hachures diagonales sur la cellule active (`FLYING FOWL`), le module `Summary` avec verrouillage des localisations (`UKRAINE`, `BLACK SEA`), et un calque SVG qui calcule en temps réel les coordonnées exactes à l'écran pour tracer la flèche pointillée reliant le mot surligné dans le texte à la cellule correspondante.

Fichier : static/index.html (Partie 1/2 — Structure et Styles CSS Tactiques)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>SIGINT Voice Analyzer — Tactical FUI</title>
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; }
  body {
    background-color: #161819;
    color: #b8c4c2;
    font-family: 'Courier New', Consolas, monospace;
    height: 100vh;
    overflow: hidden;
    display: grid;
    grid-template-columns: 58% 42%;
    padding: 28px 36px;
    position: relative;
  }
  /* Calque SVG pour les lignes pointillées de liaison */
  #link-layer {
    position: fixed;
    inset: 0;
    width: 100vw;
    height: 100vh;
    pointer-events: none;
    z-index: 20;
  }
  /* Panneau gauche : Transcription en streaming */
  .transcript-pane {
    padding-right: 32px;
    display: flex;
    flex-direction: column;
    justify-content: flex-start;
    gap: 11px;
    font-size: 16px;
    letter-spacing: 0.03em;
    overflow-y: auto;
  }
  .transcript-line {
    color: #aeb9b7;
    line-height: 1.45;
  }
  .kw-highlight {
    background-color: #44d66c;
    color: #0b1a10;
    font-weight: 700;
    padding: 1px 6px;
    display: inline-block;
  }
  /* Panneau droit : Matrice + Summary + Voice Analyzer */
  .right-pane {
    display: flex;
    flex-direction: column;
    justify-content: space-between;
    gap: 18px;
  }
  .matrix-grid {
    display: grid;
```

Fichier : static/index.html (Partie 2/2 — DOM, Client WebSocket et Traceur SVG)

```
<body>
  <svg id="link-layer">
    <defs>
      <marker id="arrowhead" markerWidth="8" markerHeight="6" refX="7" refY="3"
orient="auto">
        <polygon points="0 0, 8 3, 0 6" fill="#78a890" />
      </marker>
    </defs>
    <line id="tactical-link" x1="0" y1="0" x2="0" y2="0"
          stroke="#78a890" stroke-width="1.5" stroke-dasharray="4,4"
          marker-end="url(#arrowhead)" visibility="hidden" />
  </svg>

  <div class="transcript-pane" id="transcript"></div>

  <div class="right-pane">
    <div class="matrix-grid" id="matrix"></div>

    <div class="tactical-window">
      <div class="win-titlebar"><span></span><span>Summary</span><span>- □ x</span>
</div>
      <div class="win-banner"></div>
      <div class="meta-row">
        <div><div class="meta-label">Subject:</div><div class="meta-val" id="sum-
subj">---</div></div>
        <div><div class="meta-label">Algorithm:</div><div class="meta-val" id="sum-
algo">Key words</div></div>
        <div><div class="meta-label">Location:</div><div class="meta-val" id="sum-
loc">Scanning</div></div>
        <div><div class="meta-label">ID:</div><div class="meta-val" id="sum-
id">Pending</div></div>
      </div>
      <div id="loc-containers">
        <div class="loc-box">AWAITING LOCK</div>
      </div>
    </div>

    <div class="tactical-window">
      <div class="win-titlebar"><span>Voice Analyzer</span><span>- □ x</span></div>
      <div class="win-banner"></div>
    </div>
  </div>

  <script>
    const matrixEl = document.getElementById('matrix');
    const transcriptEl = document.getElementById('transcript');
    const svgLine = document.getElementById('tactical-link');
    let trackedSpanId = null;
    let trackedCellId = null;

    // Initialisation des 6x2 cellules de la grille supérieure
    for (let r = 0; r < 6; r++) {
      for (let c = 0; c < 2; c++) {
        const cell = document.createElement('div');
        cell.className = 'matrix-cell';
        cell.id = `cell-${r}-${c}`;
        matrixEl.appendChild(cell);
      }
    }
```

6. Procédure de Lancement et Validation

Le système peut être exécuté immédiatement en mode démonstration (qui rejoue la séquence d'interception complète et verrouille les cibles) ou branché sur votre microphone physique en temps réel :

1. Installation des paquets :
`pip install -r requirements.txt`

2. Démarrage en mode Simulation Tactique (sans microphone requis) :
`uvicorn server:app --reload --port 8000`

3. Démarrage en mode Interception Microphone Live (Silero VAD + Faster-Whisper) :
`USE_MIC=1 uvicorn server:app --port 8000`

4. Accès à l'interface FUI : Ouvrez `http://localhost:8000` dans votre navigateur pour observer la transcription, le surlignage vert et le verrouillage des panneaux en direct.